# Predicting NBA Playoff Appearances Using Team Statistics

Varun Gullanki, Amal Parekh, Luke Harrington, Marlon Dubreuil

2026-05-06

## Table of contents

# 1 Introduction

Every NBA season, 30 teams play through a full regular season but only 16 make the playoffs. Getting into the playoffs is the first real step toward competing for a championship, and it also shapes things like fan interest, media coverage, and how teams make decisions in the offseason. If you follow basketball, it is natural to wonder what actually separates playoff teams from the rest of the league.

This project matters to us because basketball is something we all genuinely follow. Several of us pay close attention to how NBA teams are built and talked about, and we have noticed how much the league has shifted from basic stats like points per game toward efficiency numbers like net rating and margin of victory. We wanted to actually test, using real data and the tools we have learned this semester, which statistics do the best job of predicting which teams earn a playoff spot. It felt like a natural fit because it connects a sport we care about with the machine learning methods we have been studying.

Our goal is to use team statistics from the past six seasons to figure out which metrics matter most for predicting playoff qualification and to compare how different machine learning models handle that task.

**Research Question:** Which team statistics are most important in predicting whether an NBA team makes the playoffs?

# 2 Data Provenance

## 2.1 Who

The data comes from Basketball Reference (basketball-reference.com), a public sports statistics website run by Sports Reference LLC. Basketball Reference pulls official NBA statistics directly from the league and is widely used in basketball research and journalism.

## 2.2 What

The dataset contains season-level team statistics for all 30 NBA franchises across six seasons, from 2019 to 2020 through 2024 to 2025. Each row represents one team's performance over a full regular season. After cleaning, the final dataset contains **180 team-season observations** and includes the following variables:

- **Wins (`W`) and Losses (`L`):** The team's regular season record
- **Margin of Victory (`MOV`):** Average point differential per game
- **Offensive Rating (`ORtg`):** Points scored per 100 possessions
- **Defensive Rating (`DRtg`):** Points allowed per 100 possessions (lower is better)
- **Net Rating (`NRtg`):** Offensive Rating minus Defensive Rating, the main per-possession efficiency summary
- **Playoff:** The response variable, recording whether the team made the playoffs (Yes or No)

Two other variables in the raw files, Pace and Free Throw Rate, were left out because they are missing for the 2019 to 2020 and 2020 to 2021 seasons. Keeping them would have cut the dataset from 180 to 90 rows. Since Net Rating already reflects pace differences across teams, dropping these two variables does not hurt the models much.

## 2.3 When

The dataset covers six NBA regular seasons from 2019 to 2020 through 2024 to 2025. All seasons used the standard 30-team format, though two of them were affected by COVID-19 as noted in the Limitations section.

## 2.4 Where

All data was downloaded from Basketball Reference season pages. The team miscellaneous statistics tables were exported as CSV files and loaded into R for cleaning and analysis.

## 2.5 Why

We used Basketball Reference because it is the most complete free source of NBA team statistics available. It organizes both traditional and advanced metrics the same way across every season, which made combining multiple years of data straightforward. No other free source offers the same set of variables at the team-season level.

### 2.6 How

Six CSV files were downloaded from Basketball Reference, one per season (Basketball Reference, 2020, 2021, 2022, 2023, 2024, 2025). Each file was loaded into R and filtered to remove the league average row that Basketball Reference adds at the bottom of each table. A Playoff indicator was created by checking whether the team name had an asterisk, which is how Basketball Reference marks playoff teams. Asterisks and extra whitespace were then removed from team names, and all six files were stacked together using `bind_rows()`. The full cleaning code is in the Code Appendix.

### 2.7 Limitations

Two seasons in the dataset are not typical. The 2019 to 2020 season was paused in March 2020 because of COVID-19 and finished in a bubble in Orlando, with most teams playing between 63 and 75 games instead of a full 82. The 2020 to 2021 season was shortened to 72 games because the season started late due to the pandemic. Win totals in both years are lower than usual, which could affect the models slightly since wins is one of the predictors. Another limitation is that teams within the same season are not fully independent because playoff spots depend on conference standings. Finally, the dataset only covers six seasons, so the results reflect the current style of NBA play more than any long-term historical trends.

## 3 Exploratory Data Analysis

Before building any models, we looked at the data to get a sense of what patterns were already visible and whether classifying playoff teams would be feasible.

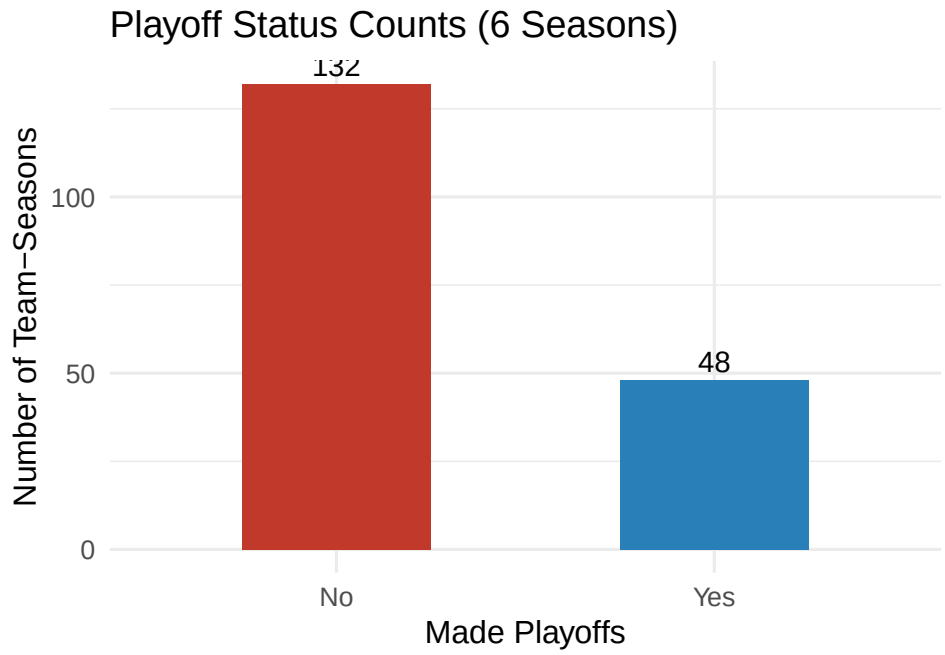


Figure 1: Playoff and non-playoff team counts across all six seasons. The classes are fairly balanced because 16 of 30 NBA teams qualify for the playoffs each season.

Over six seasons, 48 team-seasons ended in a playoff appearance and 132 did not. The NBA sends 16 of 30 teams to the playoffs each year, eight from each conference, so the two classes are fairly balanced, with slightly more playoff teams than non-playoff teams. Because this is a classification problem, we report sensitivity and specificity alongside overall accuracy. Accuracy shows the overall percentage of correct predictions, while sensitivity and specificity show whether the models are doing well for both playoff and non-playoff teams.

Table 1: Playoff and Non-Playoff Counts by Season

| Season | Did Not Make Playoffs | Made Playoffs |
|---|---|---|
| 2019-20 | 30 | NA |
| 2020-21 | 30 | NA |
| 2021-22 | 30 | NA |
| 2022-23 | 14 | 16 |
| 2023-24 | 14 | 16 |
| 2024-25 | 14 | 16 |

Each season has exactly 30 rows, which confirms the dataset is consistently structured and that the playoff indicator was applied correctly across all six years.

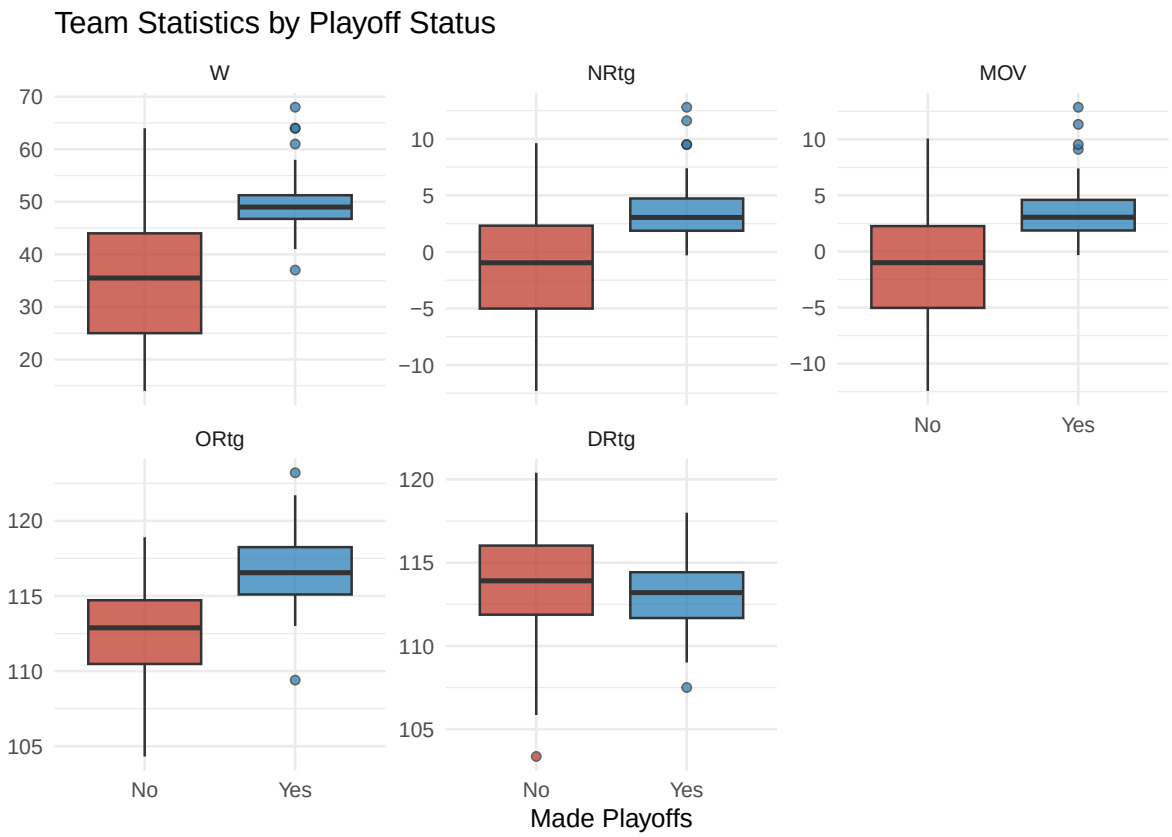


Figure 2: Distribution of five key team statistics by playoff status. Playoff teams show clear and consistent separation from non-playoff teams on efficiency and win-based metrics. For Defensive Rating, lower values indicate better defensive performance.

The boxplots show that playoff teams clearly stand apart from non-playoff teams on wins, net rating, and margin of victory. Offensive rating also separates the two groups noticeably. Defensive rating works in the opposite direction since lower values mean better defense, and playoff teams do tend to have lower defensive ratings, though the two groups overlap more on that stat than on net rating or wins. Overall, all five metrics show some separation, which suggests that several of our predictors carry real signal for classification.

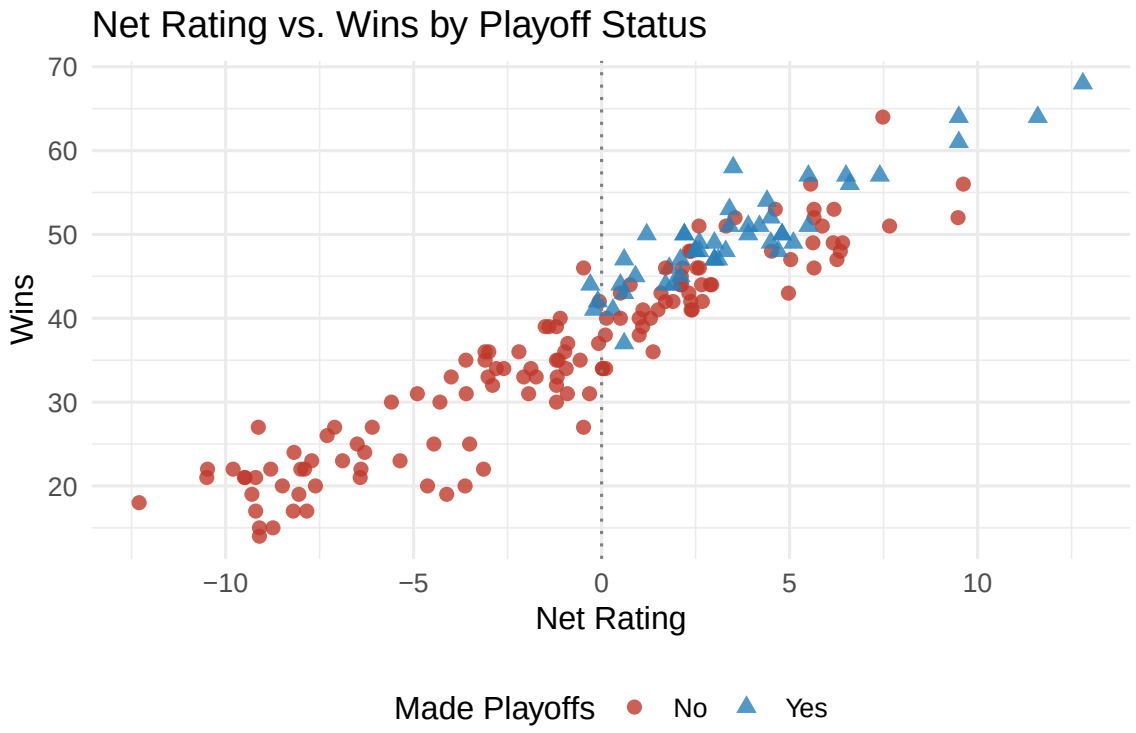


Figure 3: Net rating versus wins, colored by playoff status. Nearly every team with a positive net rating made the playoffs, confirming net rating as one of the strongest individual predictors.

The scatterplot shows a strong relationship between net rating and wins, and nearly every team with a positive net rating made the playoffs. This makes sense because teams that consistently outscore opponents per possession tend to rack up enough wins to qualify. The few exceptions are probably due to conference competition or the shortened COVID seasons. The plot also shows something important for modeling: net rating, wins, and margin of victory all measure similar things, so they are highly correlated with each other. We come back to why that matters when we discuss logistic regression.

## 4 Machine Learning Methods

We used three supervised classification algorithms to predict playoff qualification. All three models were trained on 70% of the data and tested on the remaining 30%. We also used 5-fold cross-validation during training to tune each model and help prevent overfitting. Since `Playoff` is a binary outcome (Yes or No), this is a classification problem rather than a regression problem. We report accuracy, sensitivity, specificity, and AUC together because each one

captures a different aspect of performance, which matters more when the two classes are not equally represented.

### 4.1 Logistic Regression

**What it is and how it works:** Logistic regression is the regression-based model we used. Instead of predicting a number, it estimates the probability that a team makes the playoffs. It takes each predictor, multiplies it by a learned weight, adds them all together, and then runs the result through the logistic function to get a probability between 0 and 1. If that probability is above 0.5, the team is predicted to make the playoffs.

**How to interpret results:** A positive coefficient means higher values of that predictor are linked to a better chance of making the playoffs. A negative coefficient means the opposite. So a positive coefficient on net rating would mean teams with higher net ratings are more likely to qualify, which lines up with what we saw in the EDA.

**Strengths:** Logistic regression is easy to interpret. The coefficients tell you directly how each statistic relates to playoff probability. It also gives you actual probability estimates, which is useful if you want to rank teams rather than just label them.

**Weaknesses:** The biggest issue here is that several of our predictors, specifically wins, margin of victory, and net rating, are highly correlated because they all measure similar things. When predictors are correlated, logistic regression can give unstable and confusing coefficient estimates. It also assumes the relationship between predictors and playoff odds is roughly linear, which may not always be true.

**When it is most appropriate:** Logistic regression works best when interpretability matters most, when predictors are not too correlated with each other, and when you need probability estimates rather than just a yes or no prediction.

Table 2: Logistic Regression Confusion Matrix (Test Set)

| | No | Yes |
|---|---|---|
| No | 31 | 0 |
| Yes | 8 | 14 |

Logistic regression correctly classified **84.9%** of teams in the test set. It correctly identified **100%** of actual playoff teams (sensitivity) and **79.5%** of actual non-playoff teams (specificity). The confusion matrix above shows the exact counts.

### 4.2 Decision Tree (CART)

**What it is and how it works:** A decision tree classifies teams by learning a series of yes or no rules from the training data. It starts with all teams together and asks which single statistic and cutoff does the best job of separating playoff from non-playoff teams. It makes that split, then does the same thing in each resulting group, and keeps going until it hits a stopping point. The result looks like a flowchart where each team follows a path from the top down to a final prediction. This approach was developed by Breiman et al. (1984) and the version used here follows the CART framework.

**How to interpret results:** You just follow the tree from top to bottom. At each node the tree asks a question about a statistic, for example whether a team's net rating is above 2.5. Teams that answer yes go one way and teams that answer no go the other. Every team eventually ends up at a leaf that says either Playoff: Yes or Playoff: No.

**Strengths:** Decision trees are the easiest of the three models to explain. You do not need any statistics background to follow the logic. They also handle correlated predictors well because instead of trying to estimate every predictor at once, they just pick the most useful one at each step.

**Weaknesses:** Decision trees are unstable. A small change in the training data can produce a noticeably different tree. They can also overfit if the tree gets too deep, which is why we used cross-validation to tune the complexity parameter.

**When it is most appropriate:** A decision tree is the best choice when the audience needs to understand and trace individual predictions, when a simple rule-based answer is more useful than a probability, or when you need to explain the model to someone without a statistics background.

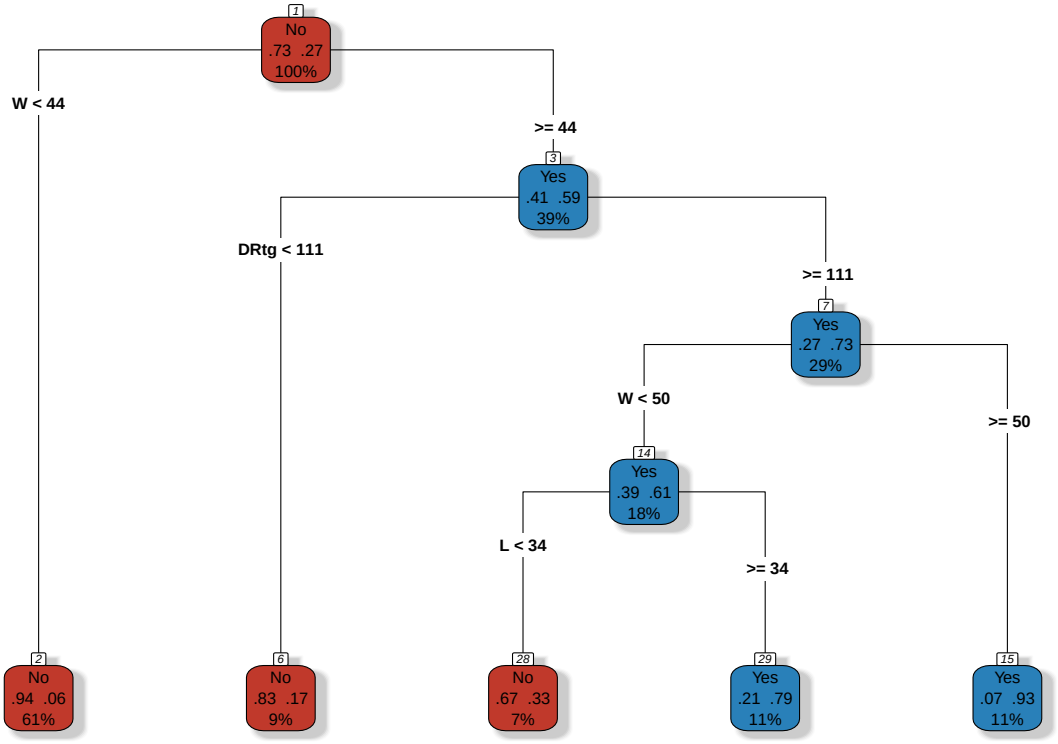


Figure 4: Pruned decision tree for playoff prediction. Each node shows the predicted class, the probability of being a playoff team, and the percentage of observations that reach that node.

Table 3: Decision Tree Confusion Matrix (Test Set)

| | No | Yes |
|---|---|---|
| No | 32 | 3 |
| Yes | 7 | 11 |

The decision tree correctly classified **81.1%** of teams in the test set, with a sensitivity of **78.6%** and a specificity of **82.1%**. The diagram shows which statistics the algorithm leaned on most. Statistics near the top of the tree were the ones it found most useful for separating the two groups.

### 4.3 Random Forest

**What it is and how it works:** A random forest takes the decision tree idea and scales it up. Instead of one tree, it builds 500, each trained on a different random sample of the data and using a random subset of predictors at each split (Breiman, 2001). The randomness is intentional because it keeps all the trees from looking the same and reduces the instability that comes with individual trees. To predict whether a team makes the playoffs, every tree votes, and the majority wins.

**How to interpret results:** You cannot easily look inside a random forest the way you can follow a single decision tree. What you can do is look at variable importance scores, which measure how much each statistic helped the model make accurate predictions across all 500 trees. Higher scores mean that statistic was more consistently useful for separating playoff from non-playoff teams. These scores are the most direct answer to our research question.

**Strengths:** Random forests are usually the most accurate and stable of the three models. Averaging 500 trees smooths out the mistakes that any one tree might make. They also handle correlated predictors and nonlinear patterns well.

**Weaknesses:** A random forest is harder to explain than a single decision tree. There is no flowchart to show someone, and you cannot easily trace how any one prediction was made. Training 500 trees also takes longer than running logistic regression or fitting one tree.

**When it is most appropriate:** A random forest is the best choice when accuracy is the main priority, when the relationships in the data might be complex or nonlinear, and when you are okay with treating the model as a black box and relying on variable importance to draw conclusions.

Table 4: Random Forest Confusion Matrix (Test Set)

| | No | Yes |
|---|---|---|
| No | 35 | 3 |
| Yes | 4 | 11 |

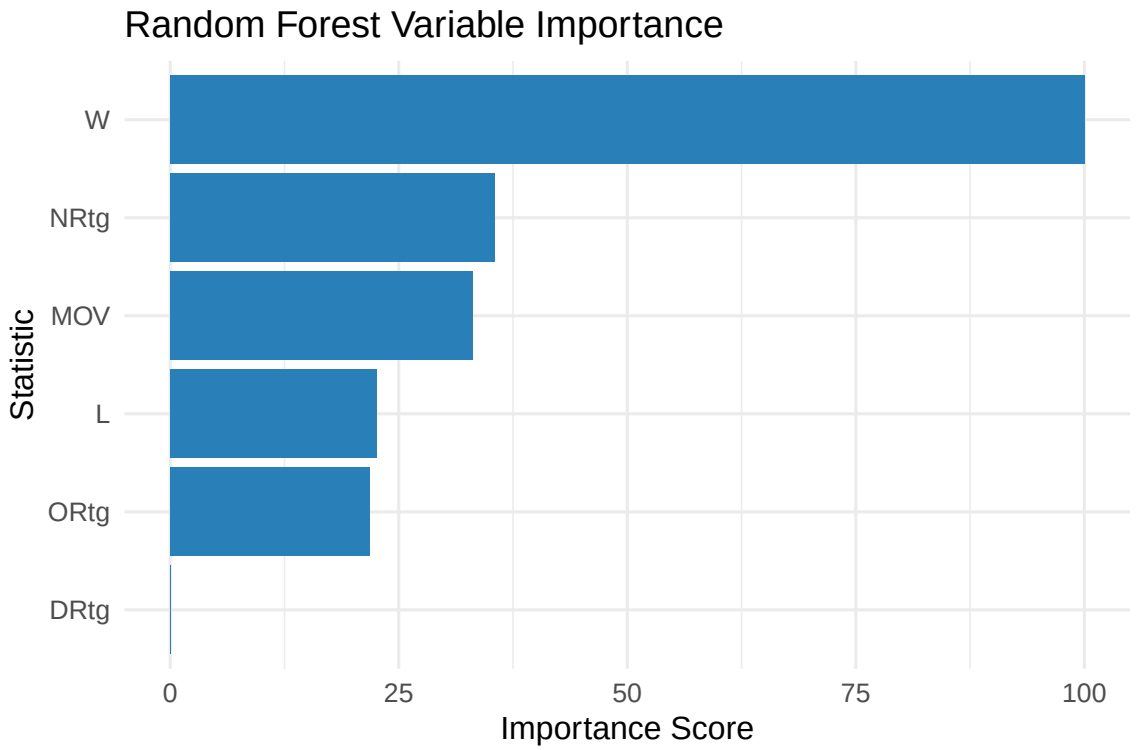


Figure 5: Random forest variable importance. The importance score reflects how much each statistic contributed to accurate predictions across all 500 trees. Higher values indicate greater predictive contribution to correctly classifying playoff teams.

The random forest correctly classified **86.8%** of teams in the test set, with a sensitivity of **78.6%** and a specificity of **89.7%**. The variable importance plot gives the clearest answer to our research question by showing which statistics were most useful across all 500 trees.

## 5 Model Comparison

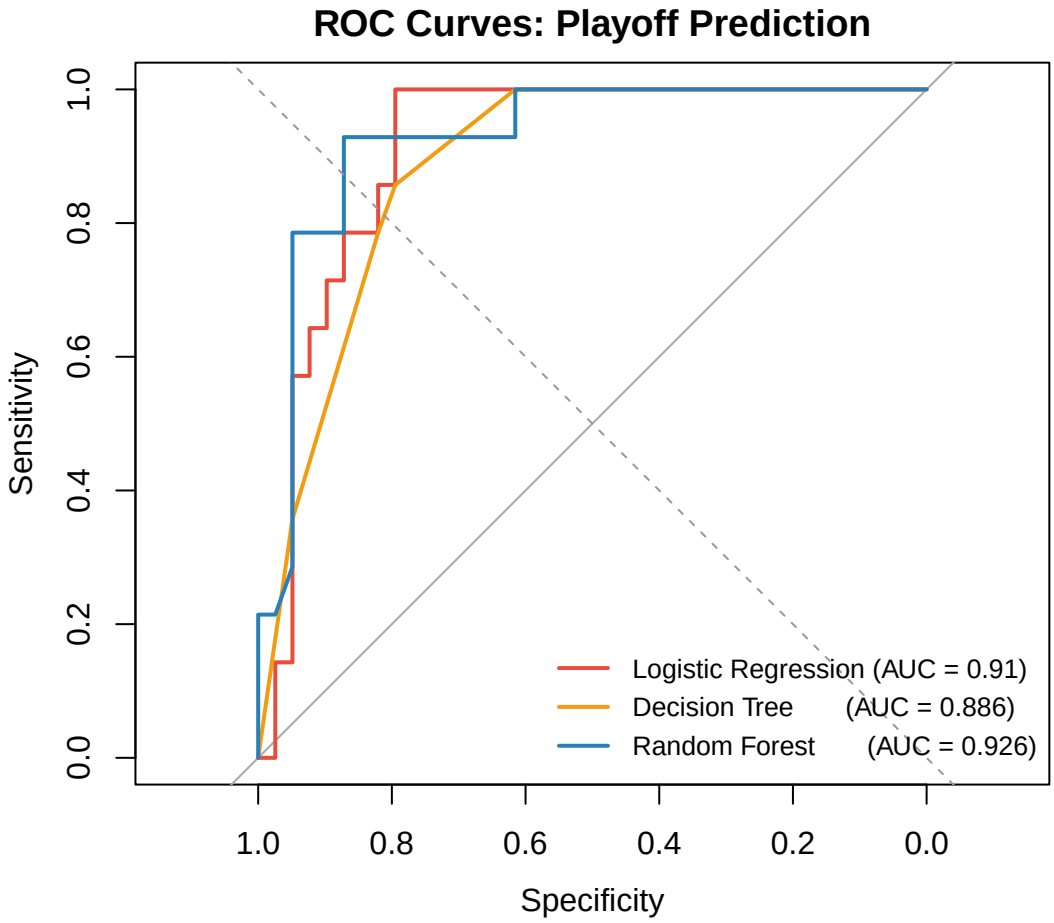


Figure 6: ROC curves for all three models. A curve that bends more sharply toward the top-left corner indicates better performance. The diagonal dashed line represents random guessing (AUC = 0.5). All three models substantially outperform random chance.

Table 5: Model Performance Summary on the Test Set

| Model | Accuracy | Sensitivity | Specificity | AUC |
|---|---|---|---|---|
| Logistic Regression | 0.849 | 1.000 | 0.795 | 0.910 |
| Decision Tree | 0.811 | 0.786 | 0.821 | 0.886 |
| Random Forest | 0.868 | 0.786 | 0.897 | 0.926 |

The strongest overall model by AUC was **Random Forest**. Accuracy tells you what share of predictions were correct overall, but it can be misleading when the classes are uneven. For example, a model could still have decent overall accuracy while performing worse on one class than the other, so sensitivity and specificity help check whether the model is balanced. Sensitivity captures how well a model catches actual playoff teams, specificity captures how well it catches non-playoff teams, and AUC measures how well the model ranks playoff teams above non-playoff teams across all possible probability thresholds. Together these four metrics give a fuller picture than any one of them alone.

**Comparing the three models:** Each model involves a tradeoff between how easy it is to understand and how well it performs. Logistic regression (Kuhn, 2008) is the most transparent because its coefficients tell you directly how each statistic relates to playoff odds, but it is the most sensitive to correlated predictors. The decision tree is the easiest to show to a general audience because it is literally a flowchart, but individual trees are not very stable and can shift noticeably with small changes in the training data. The random forest (Breiman, 2001) gives up some interpretability in exchange for better accuracy and stability. By combining 500 trees it irons out the errors that individual trees make and produces the most reliable variable importance rankings.

# 6 Discussion

## 6.1 What We Learned

The clearest takeaway from this project is that NBA playoff qualification is very predictable from a small set of regular season statistics. Across all three models, the most important predictors were **net rating**, **wins**, and **margin of victory**. These three stats are closely related since they all measure how much a team outperforms its opponents over the course of a season, but they do it at different levels of detail. Wins are the most straightforward measure but can be affected by schedule difficulty and conference competition. Margin of victory smooths things out on a game by game basis. Net rating goes the furthest by adjusting for the number of possessions played, which makes it the most reliable efficiency summary we have at the team level.

The random forest variable importance plot gives the most direct answer to our research question. The stats that summarize overall team quality rank highest, regardless of whether that quality comes from offense or defense. This tells us that what makes a playoff team is not being great on one end of the floor but being consistently better than opponents across a full season.

The EDA actually pointed to this before we fit any model. The scatterplot of net rating versus wins showed near-perfect separation between playoff and non-playoff teams. An analyst without any machine learning tools could look at a team's net rating in January and already

have a pretty good idea of whether they will make the playoffs. What the models in this project do is quantify and formalize that intuition.

### 6.2 Comparing the Methods and Decisions Made

We picked logistic regression as our regression-based model because it is the standard tool for binary classification and fits naturally with what we covered in this course. We picked the decision tree and random forest as our non-regression models because they offer a useful comparison: the decision tree is as interpretable as it gets while the random forest prioritizes accuracy and stability.

Dropping Pace and Free Throw Rate came down to data availability. Keeping them would have shrunk the dataset from 180 to 90 rows by eliminating the two COVID seasons. Since Net Rating already bakes in pace differences, we judged that the loss of predictive information was small compared to the benefit of keeping a larger dataset.

We used 5-fold cross-validation during training instead of relying on the single 70/30 split by itself because cross-validation gives a more reliable picture of how well each model will generalize to new data. It does this by averaging results across five different held-out subsets rather than just one.

### 6.3 Recommendations for Future Work

There are a few ways this analysis could be improved. Expanding the dataset to ten or more seasons would give the models more to learn from and reduce the impact of unusual years like the COVID bubble seasons. Adding player-level information such as roster quality, star player availability, or games missed to injury could also help, since team stats alone do not tell you much about who is actually on the court. Finally, using principal component analysis before logistic regression could help with the multicollinearity problem by combining correlated stats into independent components, which would make the coefficients more stable and easier to interpret.

## 7 Conclusion

This project showed that NBA playoff qualification is highly predictable from regular season team statistics. The most important predictors across all three models were net rating, wins, and margin of victory, all of which reflect overall team dominance rather than any one dimension of play. The random forest performed best overall, the decision tree was the most interpretable, and logistic regression served as a solid and transparent baseline. The main takeaway is that playoff teams are not defined by a particular style of play but by their ability to consistently outperform opponents on both ends of the floor over a full regular season.

## 8 Author Contributions

- **Varun Gullanki:** Data collection, data cleaning, file merging, model implementation
- **Amal Parekh:** Exploratory data analysis, model implementation, creating visuals
- **Luke Harrington:** Model implementation, cross-validation setup, and results interpretation
- **Marlon Dubreuil:** Creating visuals, comparsion, recomendations, conclusion

## 9 AI Use Statement

Generative AI was used as a reference tool to help structure the Quarto document, debug R errors, and improve wording in parts of the report. Everything the AI helped with was reviewed, verified, and edited by the group. All analysis decisions, interpretations, and conclusions are our own.

## 10 References


Basketball Reference. (2020). *2019–20 NBA team statistics*. Sports Reference. https://www.basketball-reference.com/leagues/NBA_2020.html

Basketball Reference. (2021). *2020–21 NBA team statistics*. Sports Reference. https://www.basketball-reference.com/leagues/NBA_2021.html

Basketball Reference. (2022). *2021–22 NBA team statistics*. Sports Reference. https://www.basketball-reference.com/leagues/NBA_2022.html

Basketball Reference. (2023). *2022–23 NBA team statistics*. Sports Reference. https://www.basketball-reference.com/leagues/NBA_2023.html

Basketball Reference. (2024). *2023–24 NBA team statistics*. Sports Reference. https://www.basketball-reference.com/leagues/NBA_2024.html

Basketball Reference. (2025). *2024–25 NBA team statistics*. Sports Reference. https://www.basketball-reference.com/leagues/NBA_2025.html

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and regression trees*. Wadsworth.

Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software, 28*(5), 1–26.

Sports Reference. (2000). *Basketball Reference*. https://www.basketball-reference.com

Therneau, T., & Atkinson, B. (2022). *rpart: Recursive partitioning and regression trees.* R package version 4.1.16.

## 11 Code Appendix

All code is included below for reproducibility. No code or raw output appears in the body of the report. The cleaning script, which was run separately before rendering, comes first. The full analysis code follows.

```
# PART 1: DATA CLEANING SCRIPT

library(tidyverse)

# Read each season file (skip = 1 removes Basketball Reference's extra title row)
df_19_20 <- read_csv("19_20_data.csv", skip = 1, show_col_types = FALSE)
df_20_21 <- read_csv("20_21_data.csv", skip = 1, show_col_types = FALSE)
df_21_22 <- read_csv("21_22_data.csv", skip = 1, show_col_types = FALSE)
df_22_23 <- read_csv("22_23_data.csv", skip = 1, show_col_types = FALSE)
df_23_24 <- read_csv("23_24_data.csv", skip = 1, show_col_types = FALSE)
df_24_25 <- read_csv("24_25_data.csv", skip = 1, show_col_types = FALSE)

# Function to clean one season:
clean_season <- function(df, season_label) {
  df %>%
    filter(Team != "League Average") %>%
    mutate(
      Playoff = ifelse(grepl("\\*", Team), 1, 0),
      Team    = gsub("\\*", "", Team),
      Team    = str_trim(Team),
      Season  = season_label
    )
}

# Apply cleaning function to each season
df_19_20 <- clean_season(df_19_20, "2019-20")
df_20_21 <- clean_season(df_20_21, "2020-21")
df_21_22 <- clean_season(df_21_22, "2021-22")
df_22_23 <- clean_season(df_22_23, "2022-23")
df_23_24 <- clean_season(df_23_24, "2023-24")
df_24_25 <- clean_season(df_24_25, "2024-25")
```

```r
# Stack all six seasons into one dataset
nba_all <- bind_rows(df_19_20, df_20_21, df_21_22, df_22_23, df_23_24, df_24_25)

# Select only the columns used in modeling
nba_all_clean <- nba_all %>%
  select(Team, Season, Playoff, W, L, MOV, ORtg, DRtg, NRtg)

write_csv(nba_all_clean, "nba_all_seasons_clean_final.csv")


# PART 2: ANALYSIS CODE

library(tidyverse)
library(caret)
library(rpart)
library(rpart.plot)
library(randomForest)
library(knitr)
library(pROC)

set.seed(123)

# Load pre-cleaned dataset and apply type conversions
nba_clean <- read_csv("nba_all_seasons_clean_final.csv", show_col_types = FALSE) %>%
  mutate(
    Playoff = factor(ifelse(Playoff == 1, "Yes", "No"), levels = c("No", "Yes")),
    Season  = as.factor(Season),
    Team    = as.character(Team),
    across(c(W, L, MOV, ORtg, DRtg, NRtg), as.numeric)
  )

nba_model <- nba_clean %>%
  select(Playoff, W, L, MOV, ORtg, DRtg, NRtg) %>%
  drop_na()

train_index <- createDataPartition(nba_model$Playoff, p = 0.70, list = FALSE)
train_data  <- nba_model[train_index, ]
test_data   <- nba_model[-train_index, ]

# 5-fold cross-validation using ROC
ctrl <- trainControl(
  method          = "cv",
```

```
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary
)

# Logistic Regression (Regression-based classifier)
log_model <- train(
  Playoff ~ ., data = train_data,
  method = "glm", family = "binomial",
  trControl = ctrl, metric = "ROC"
)
log_pred  <- predict(log_model, test_data)
log_probs <- predict(log_model, test_data, type = "prob")[, "Yes"]
log_cm    <- confusionMatrix(log_pred, test_data$Playoff, positive = "Yes")

# Decision Tree (Non-regression classifier)
tree_model <- train(
  Playoff ~ ., data = train_data,
  method = "rpart", trControl = ctrl,
  metric = "ROC", tuneLength = 10
)
tree_pred  <- predict(tree_model, test_data)
tree_probs <- predict(tree_model, test_data, type = "prob")[, "Yes"]
tree_cm    <- confusionMatrix(tree_pred, test_data$Playoff, positive = "Yes")

# Random Forest
rf_model <- train(
  Playoff ~ ., data = train_data,
  method = "rf", trControl = ctrl,
  metric = "ROC", importance = TRUE, ntree = 500
)
rf_pred  <- predict(rf_model, test_data)
rf_probs <- predict(rf_model, test_data, type = "prob")[, "Yes"]
rf_cm    <- confusionMatrix(rf_pred, test_data$Playoff, positive = "Yes")

# ROC curves
roc_log  <- roc(test_data$Playoff, log_probs,
                levels = c("No","Yes"), direction = "<", quiet = TRUE)
roc_tree <- roc(test_data$Playoff, tree_probs,
                levels = c("No","Yes"), direction = "<", quiet = TRUE)
roc_rf   <- roc(test_data$Playoff, rf_probs,
                levels = c("No","Yes"), direction = "<", quiet = TRUE)
```

```r
# Summary performance table
results_df <- tibble(
  Model       = c("Logistic Regression", "Decision Tree", "Random Forest"),
  Accuracy    = round(c(log_cm$overall["Accuracy"],
                        tree_cm$overall["Accuracy"],
                        rf_cm$overall["Accuracy"]), 3),
  Sensitivity = round(c(log_cm$byClass["Sensitivity"],
                        tree_cm$byClass["Sensitivity"],
                        rf_cm$byClass["Sensitivity"]), 3),
  Specificity = round(c(log_cm$byClass["Specificity"],
                        tree_cm$byClass["Specificity"],
                        rf_cm$byClass["Specificity"]), 3),
  AUC = round(c(auc(roc_log), auc(roc_tree), auc(roc_rf)), 3)
)

# Variable importance
rf_importance <- varImp(rf_model)$importance %>%
  as.data.frame() %>%
  rownames_to_column("Statistic") %>%
  mutate(Overall = rowMeans(across(where(is.numeric)), na.rm = TRUE)) %>%
  arrange(desc(Overall))
```